IMVFX HW2-2 — cLDM Report Template (2026)

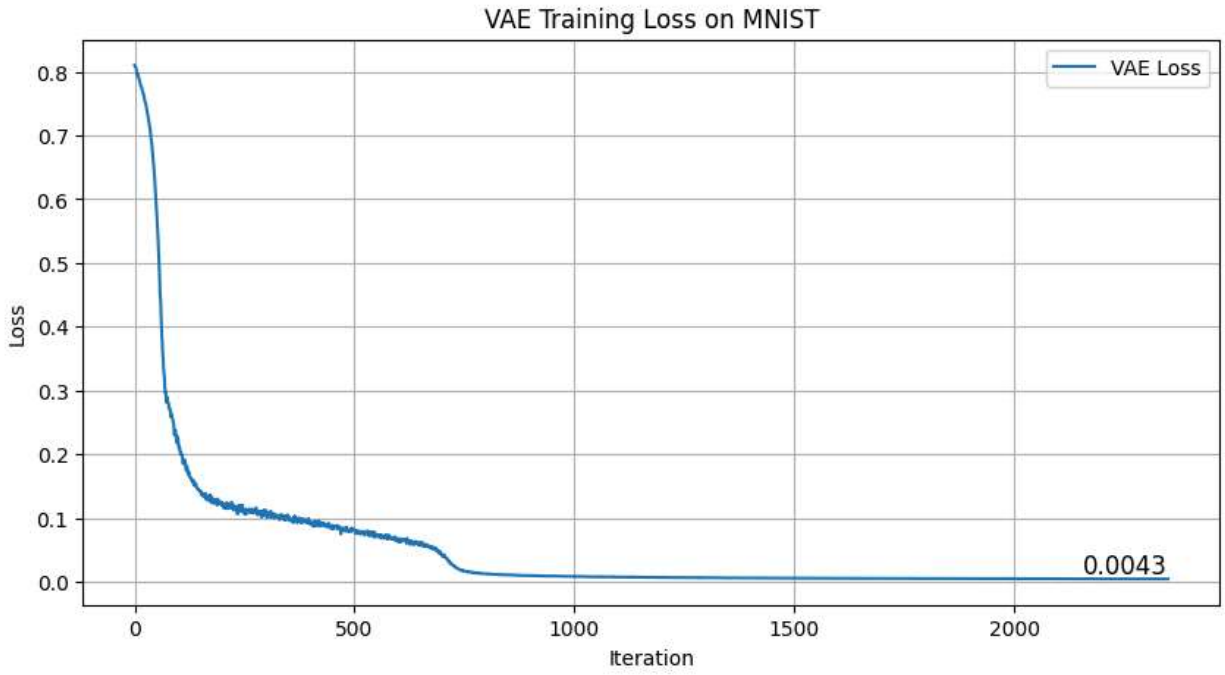
ID: 313552011, Name: 高聖傑

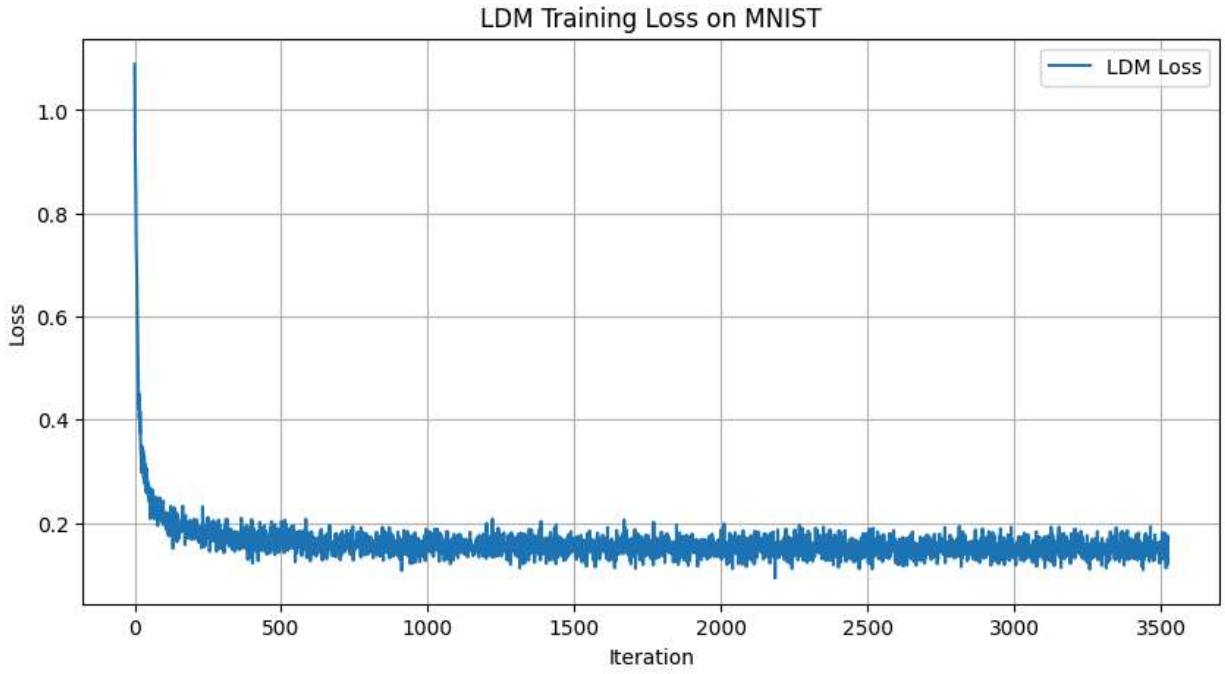
warning
To make the grading efficient, you should follow the format of the template. That is keep the structure and answer the questions.
Follow this template strictly to speed up grading and be concise, please.
Thank you.

Basic - 70%

MNIST - 30%

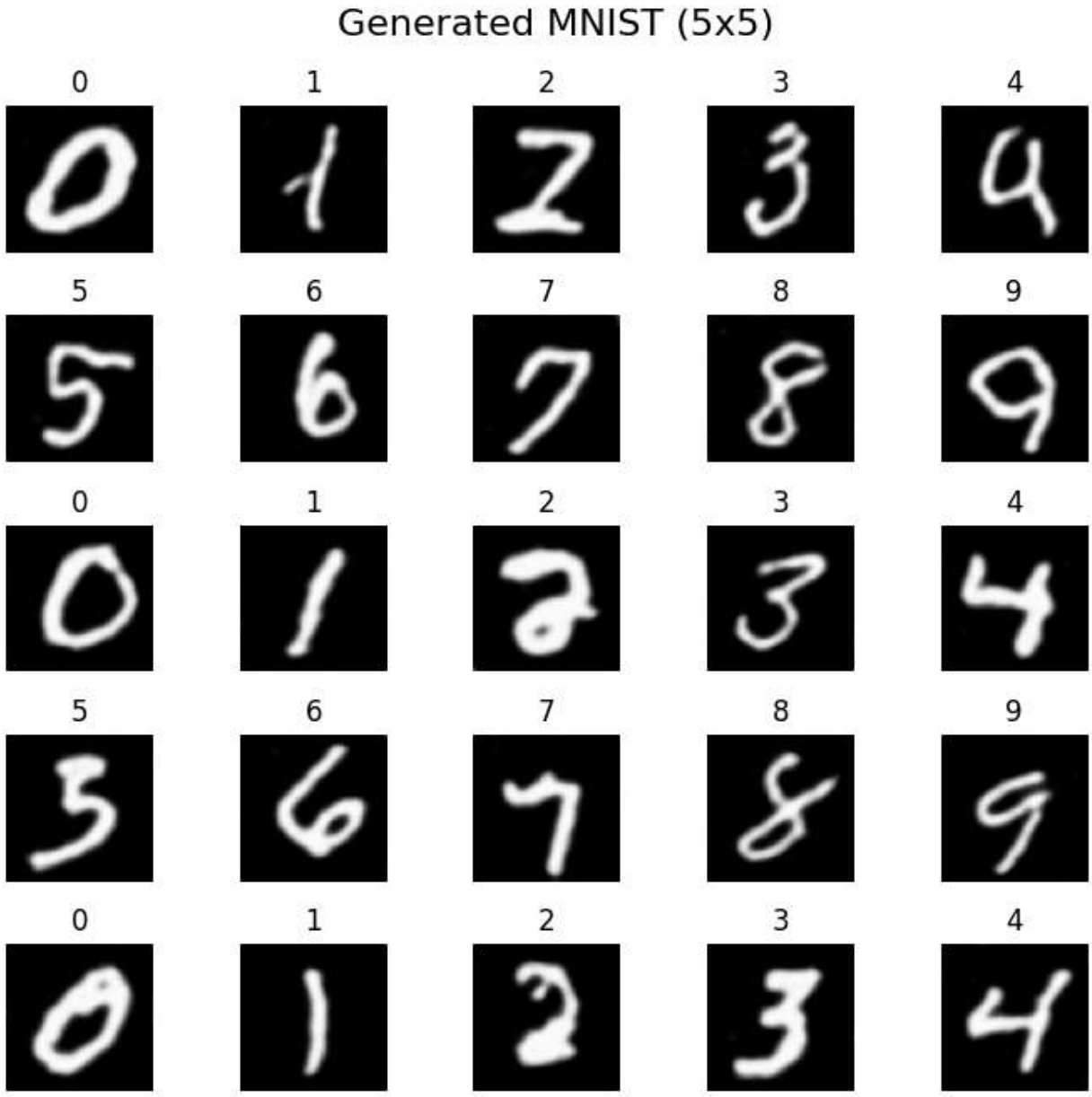
Training Loss Curve - 15%

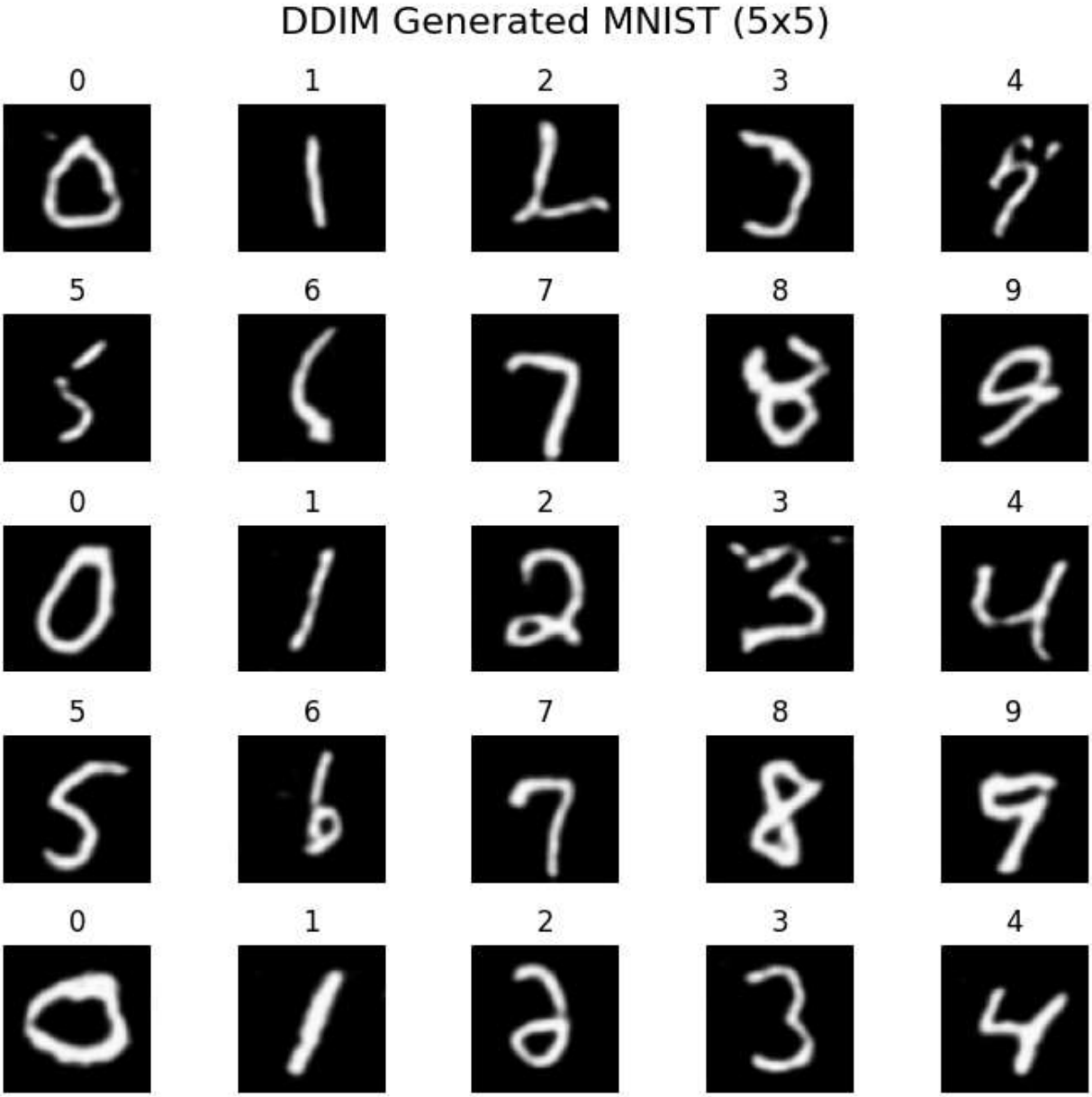




- Insert **loss curve figure**
  - x-axis: training step / epoch
  - y-axis: loss

Generated Samples (5×5) - 15%





- Insert **one 5×5 image grid**
- Each image: 64x64

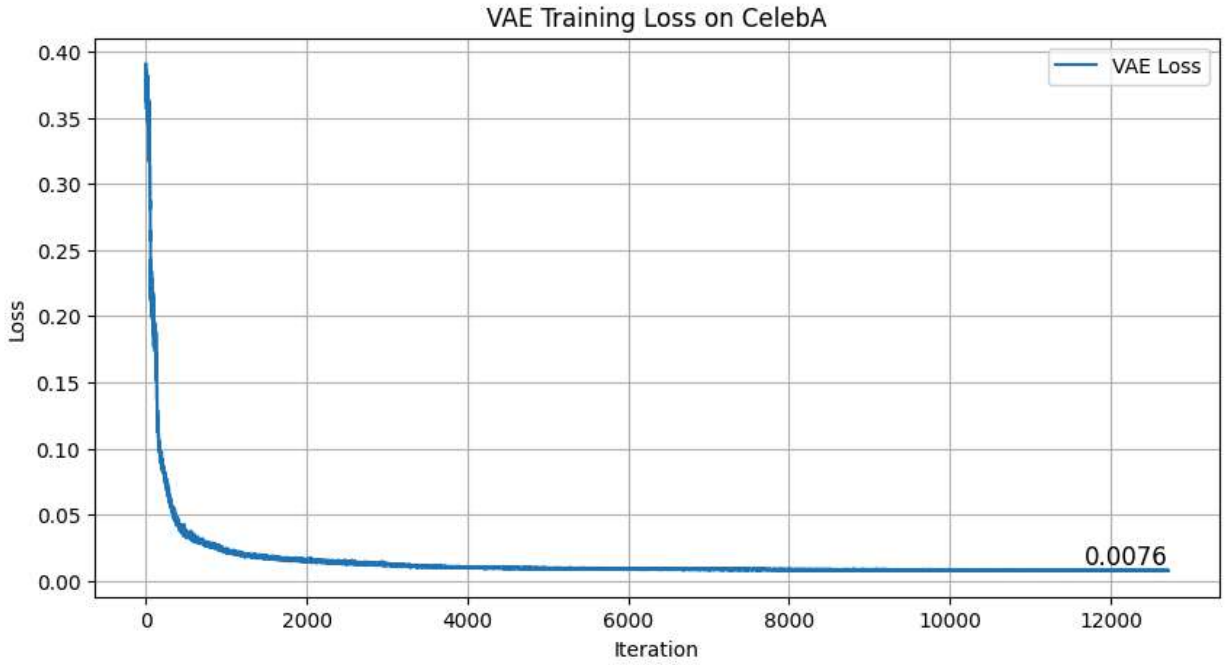
CelebA - 40%

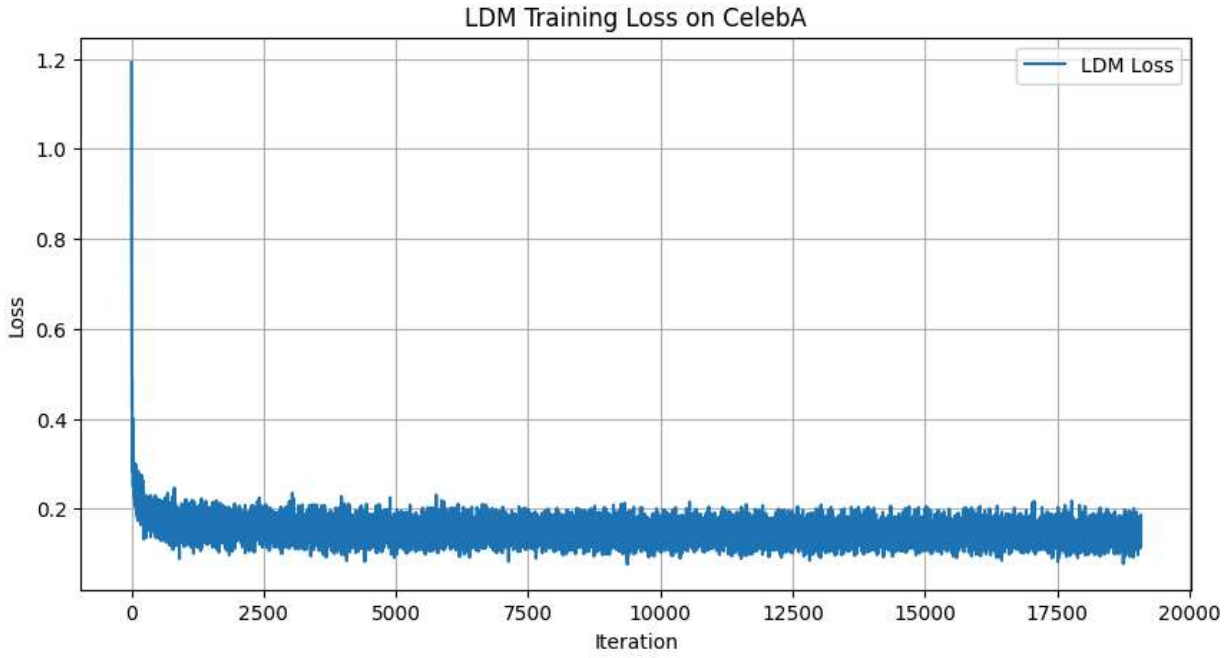
Describe your implementation idea - 10%

I implemented a Conditional Latent Diffusion Model (cLDM) customized for generating CelebA face images conditioned on the "Smiling" attribute. First, I trained a Convolutional Variational Autoencoder (VAE) to compress the $64 \times 64 \times 3$ RGB images into a lower-dimensional latent space to save compute and VRAM. Once trained, the VAE's parameters were frozen. I then trained a U-Net on these latent variables to act as the noise predictor. The conditional label (0 for Not smiling, 1 for Smiling) is embedded and injected

into the U-Net. During generation, the U-Net learns to denoise a random latent vector conditioned on the requested label, which is then mapped back to the pixel space by the VAE decoder.

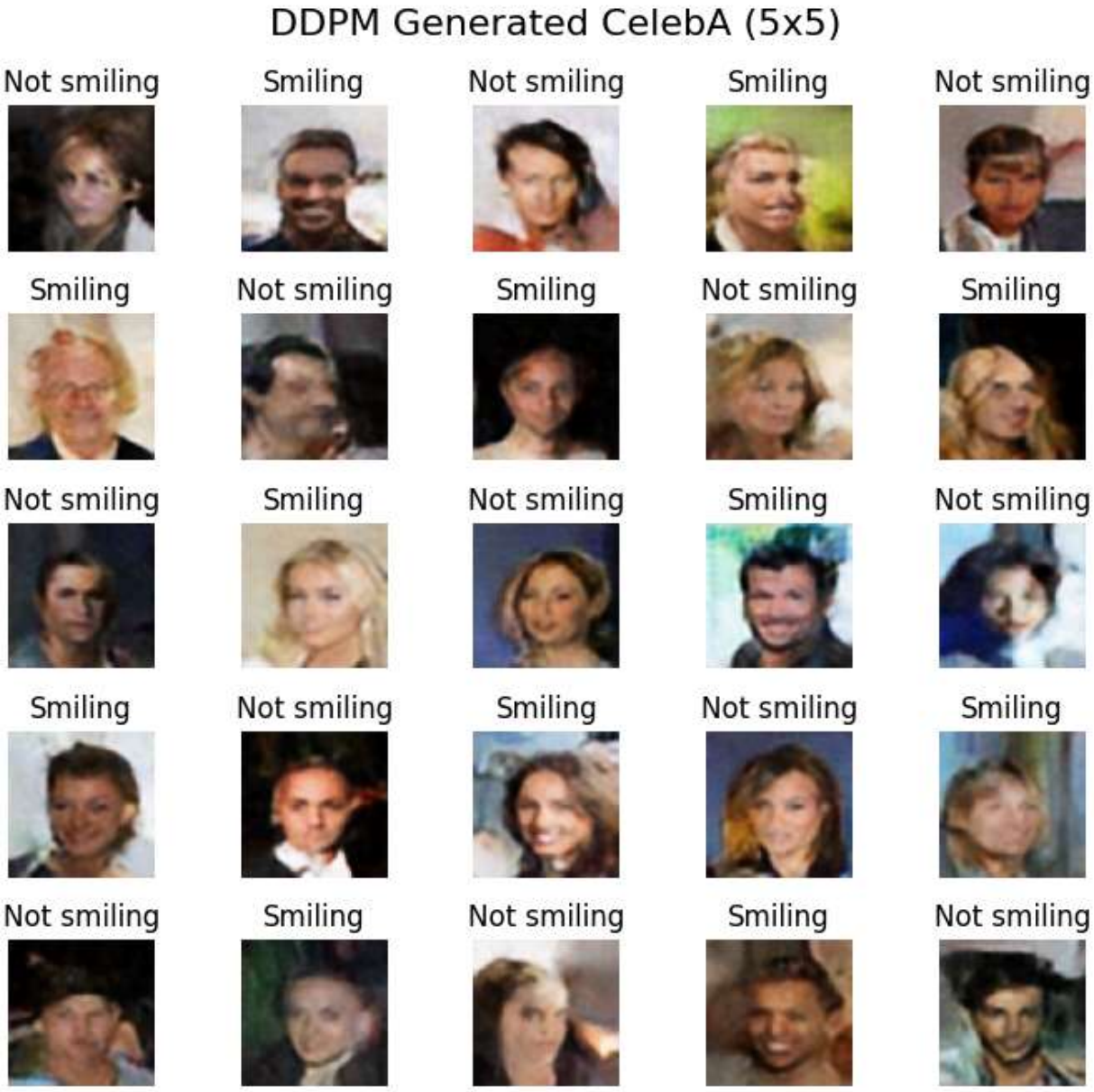
Training Loss Curve - 15%

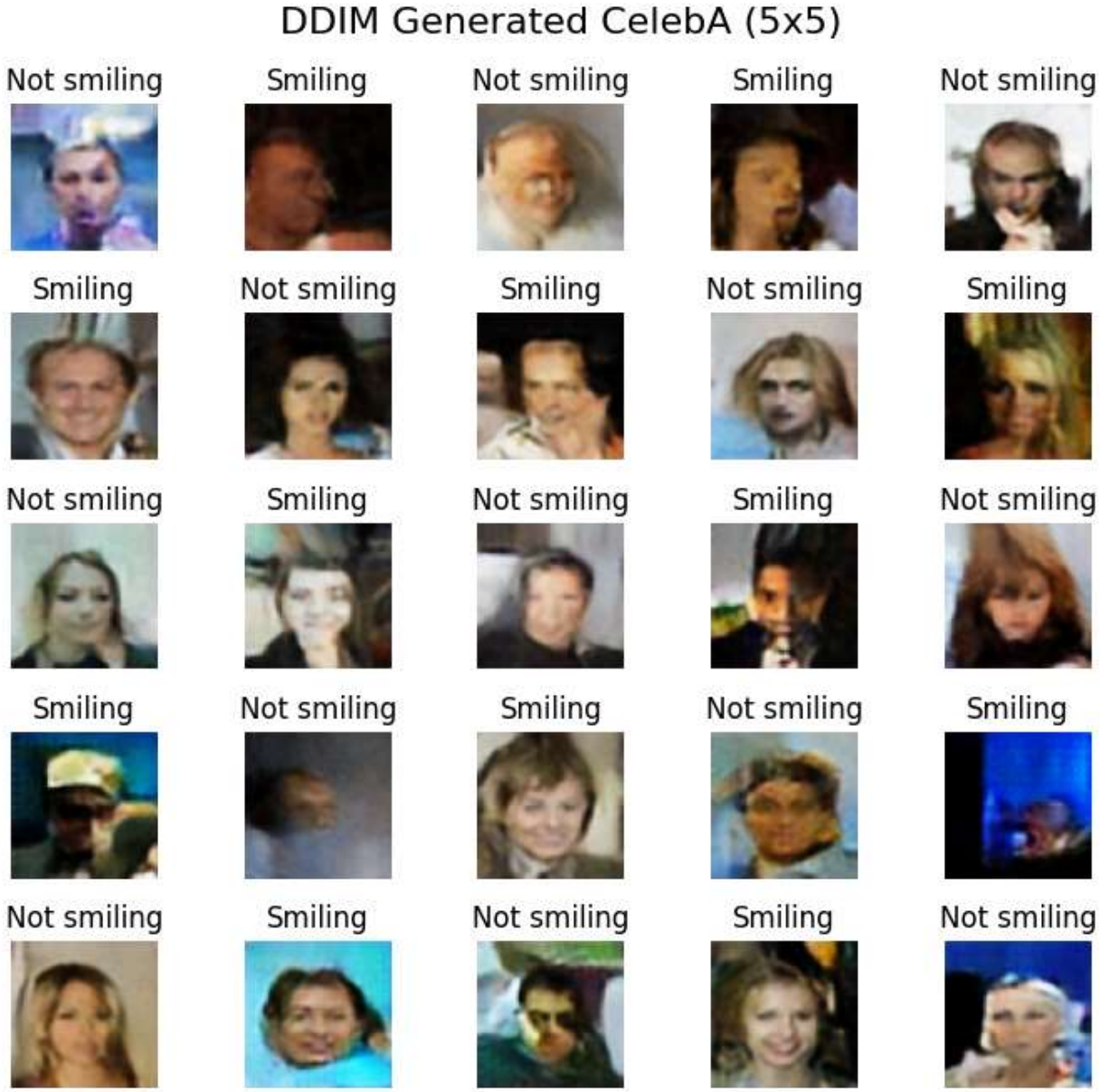




- Insert **loss curve figure**

Generated Samples (5×5) - 15%





- Insert **one 5×5 image grid**
- Each image: 64×64

Advanced - 30%

Implement DDIM and compare DDPM and DDIM - 5%

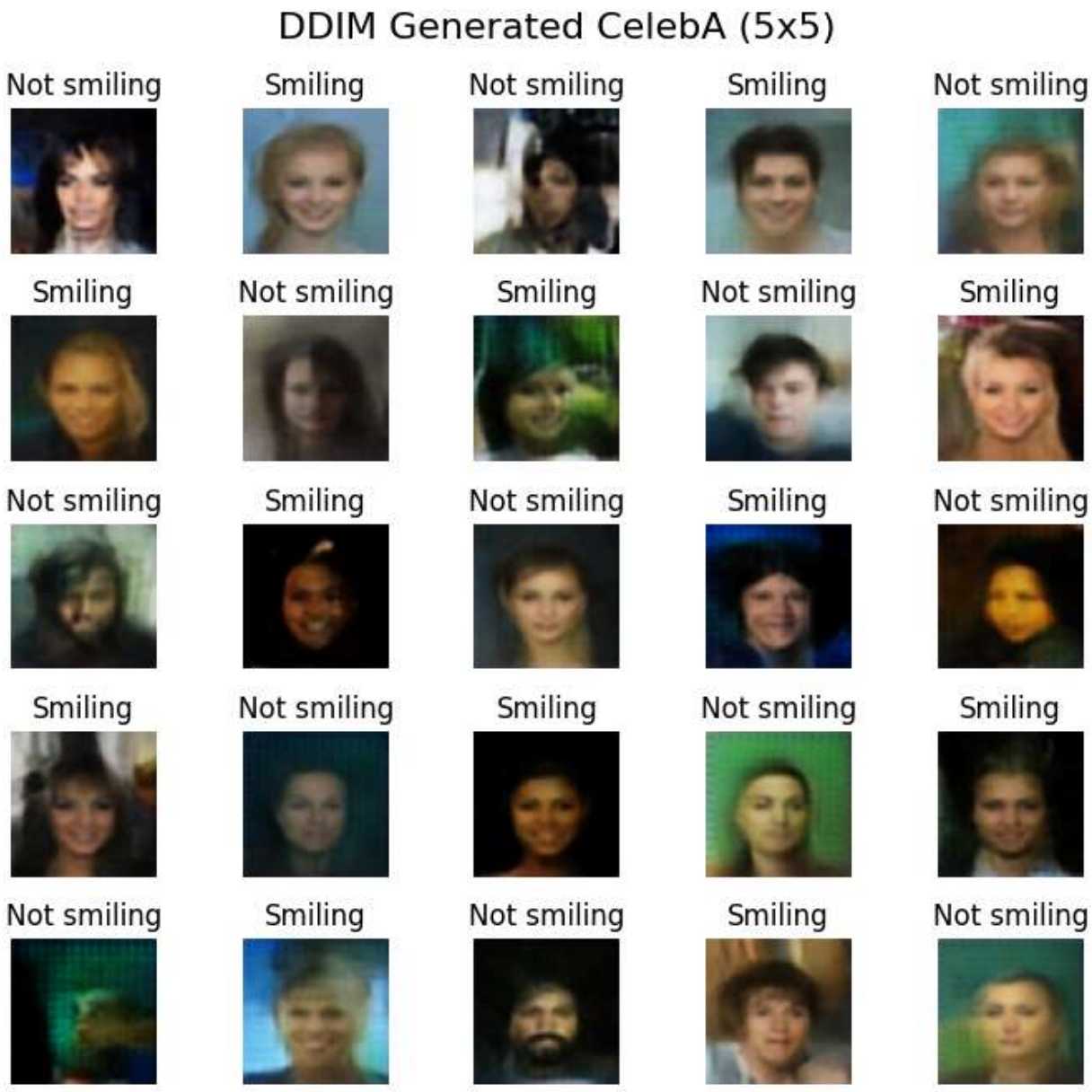
- Describe your DDIM implementation.
  I implemented the DDIM (Denoising Diffusion Implicit Models) sampling procedure within `DiffusionCore.ddim_sample` to accelerate generation. Instead of the strict Markovian generation path used in DDPM requiring all $T = 1000$ steps, DDIM computes a non-Markovian process that allows for jumping across timesteps. By taking a smaller subsequence of timesteps (e.g., 50 steps) and a variance parameter `eta` ($\eta = 0$ for deterministic sampling), it extracts an estimate of x_0 at each step to directly jump to

the previous state $x_{t-\Delta t}$.

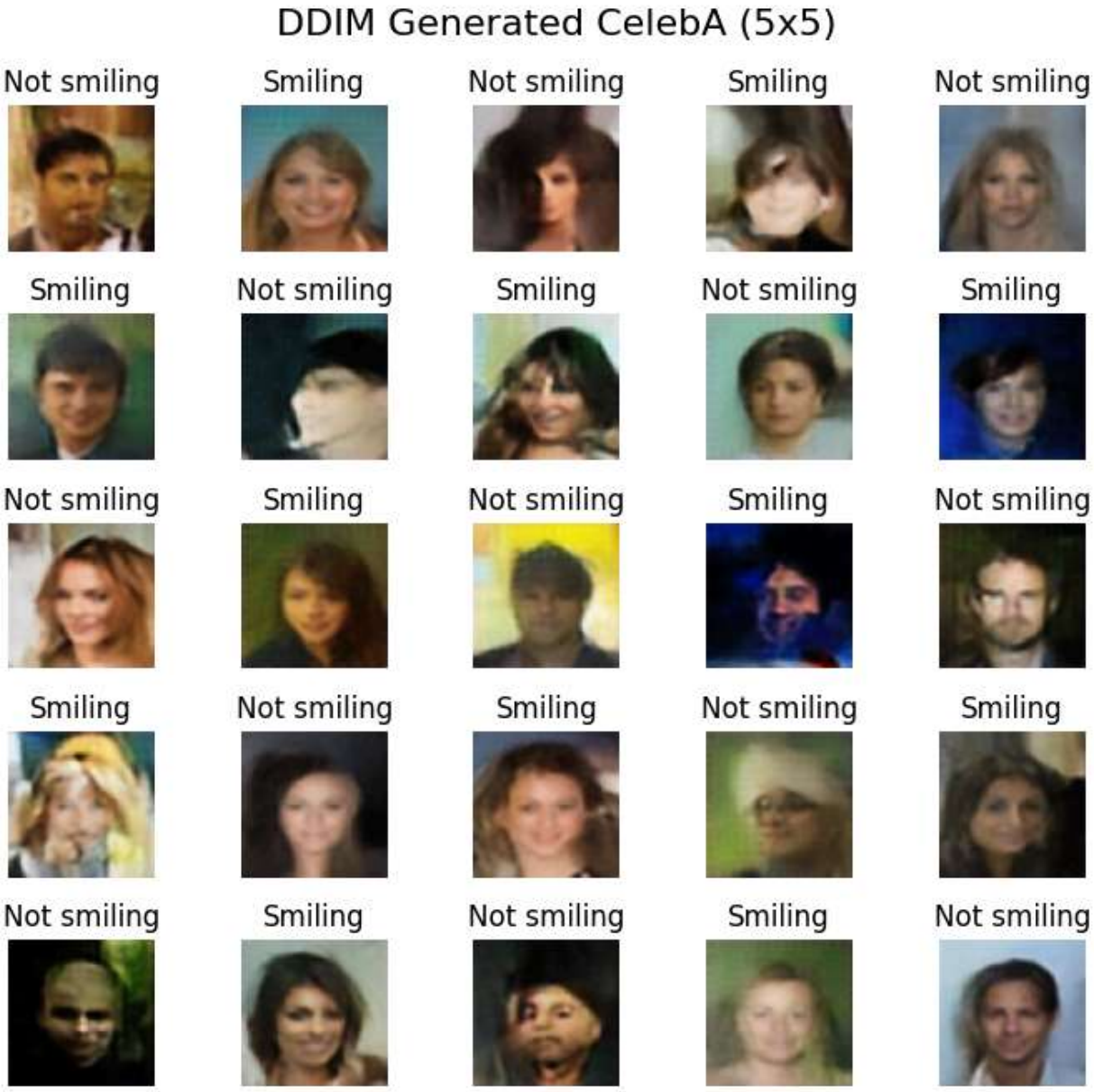
Time difference: DDIM drastically cuts down inference time compared to DDPM. While DDPM evaluates the network 1000 times, DDIM achieves similar visual quality exploring only 50 (or fewer) steps, rendering the generation pipeline ~20x faster.

- Insert **comparison figures** with description.

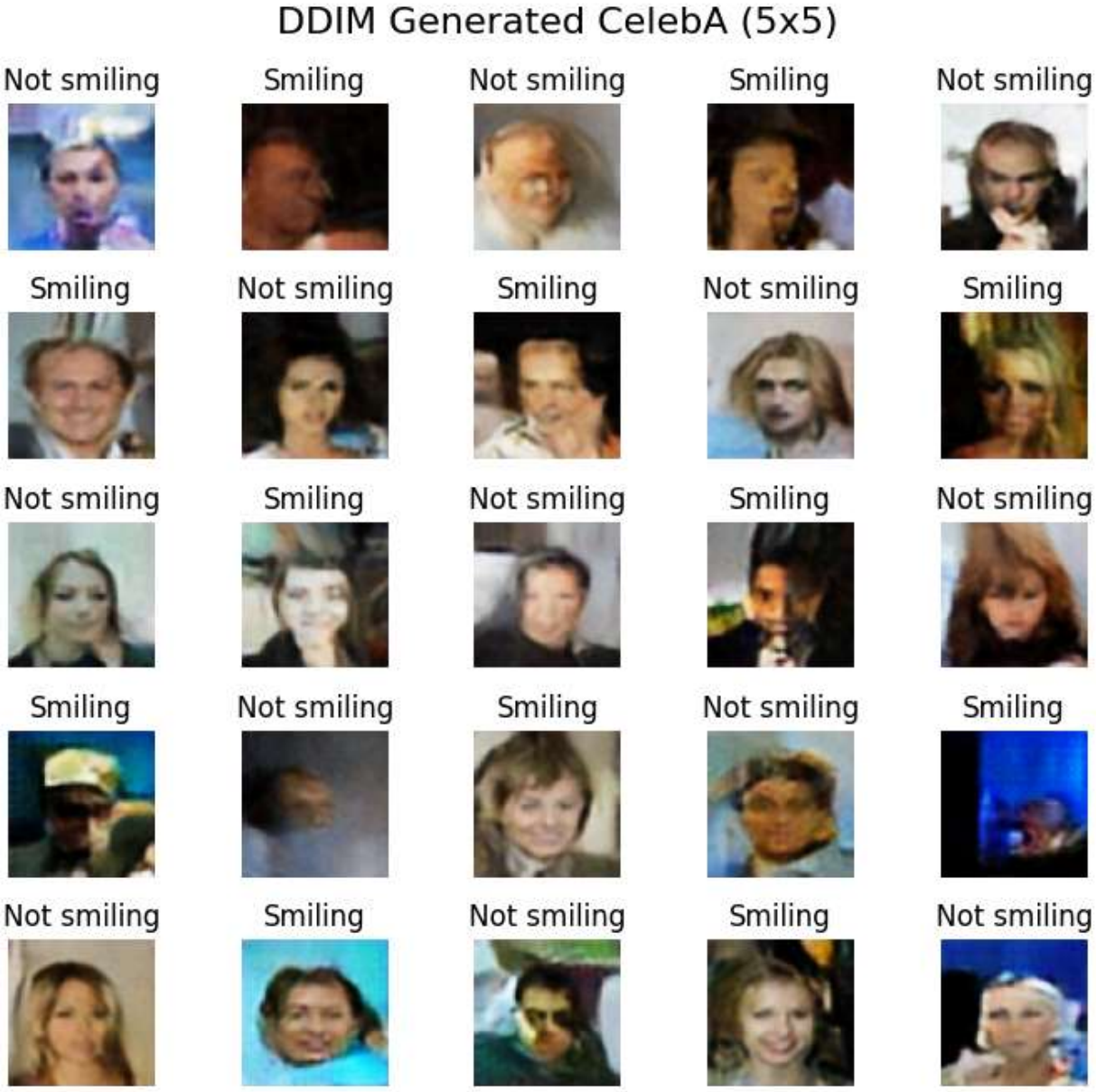
step = 5, eta = 0



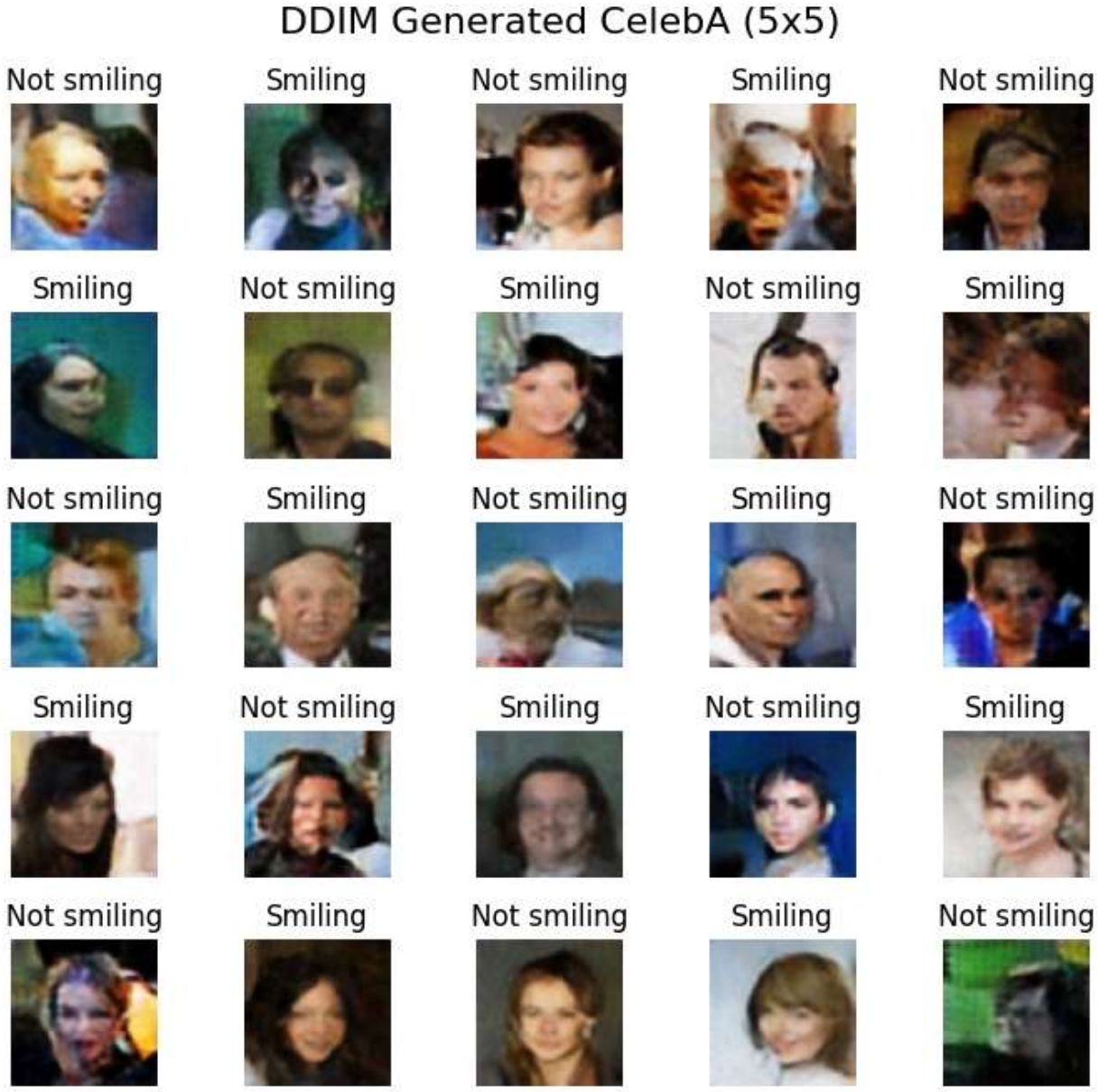
step = 10, eta = 0


DDIM Generated CelebA (5x5)
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling
Smiling
Not smiling

step = 50, eta = 0



step = 100, eta = 0



Train 5 conditions on CelebA, discuss the weakness and improvement - 5%

- Describe your implementation.
- Show the results and discuss failure cases
- Explain your improvement based on failure cases

Benchmark on CelebA - 20%

Explain the modification that make performance better.

Attempted modifications:

1. Cosine noise schedule with cutoff

```
# ---- noise schedule: linear----
betas = torch.linspace(start_beta, end_beta, n_steps)

# ---- noise schedule: cosine ----
x = torch.arange(n_steps + 1, dtype=torch.float32)

s = 0.008
alphas_cumprod = torch.cos(((x / n_steps) + s) / (1 + s) * math.pi
alphas_cumprod = alphas_cumprod / alphas_cumprod[0]
betas = 1.0 - (alphas_cumprod[1:] / alphas_cumprod[:-1])

# Clip beta to prevent math explosions
betas = torch.clip(betas, min=start_beta, max=0.9999)
```

2. Attention block

```
class AttentionBlock(nn.Module):
def __init__(self, ch: int, num_heads: int = 4):
    super().__init__()
    self.norm = nn.GroupNorm(8 if ch % 8 == 0 else 1, ch)
    self.attn = nn.MultiheadAttention(ch, num_heads, batch_first=Tr

def forward(self, x: torch.Tensor) -> torch.Tensor:
    b, c, h, w = x.shape
    h_ = self.norm(x).view(b, c, h * w).transpose(1, 2)  # (b, hw,
    attn_out, _ = self.attn(h_, h_, h_)
    attn_out = attn_out.transpose(1, 2).view(b, c, h, w)
    return x + attn_out
```

Note:
As both modifications did not make significant difference on the MNIST dataset and added time and computational cost was not worth it, I tuned the hyperparameters instead in this homework.

universal hyperparameters

```
# Diffusion
T = 1000
beta_start = 1e-4
beta_end = 0.02
lr_vae = 1e-4
lr_ldm = 1e-4

# U-Net
unet_base_ch = 64
```

MNIST hyperparameters

```
# Training
img_size = 64
batch_size = 256
num_workers = 8
n_epochs_vae = 10
n_epochs_ldm = 15


# VAE
latent_channels = 4
latent_scale = 0.18215    # was not used?
latent_hw = img_size // 4

# Conditional setup
num_classes = 10
cond_attr = None
```

CelebA hyperparameters

```
celeba_img_size = 64
celeba_batch_size = 128
celeba_n_epochs_vae = 10
celeba_n_epochs_ldm = 12
celeba_num_classes = 2
```

Fill in the table below:

Method / Setting	FID ↓
DDPM-cLDM	73.147114
DDIM-cLDM	85.001373

Notes:

- **FID**: lower is better
- **Scoring based on Ranking which is relative to classmates**